Tarea N°1 – Sistema de Análisis Robótico

EIE 434 – Programación 2 | Escuela de Ingeniería Eléctrica

Unidad: 1 **Semestre:** 1er Semestre 2026 **Docente:** Guillermo Cid Ampuero

Carreras: Ing. Electrónica, Ing. Civil Electrónica, Telecomunicaciones

Integrantes (grupos de 4): _____

Condiciones generales de entrega

Modalidad:	Grupal – 3 - 4 integrantes por grupo
Plazo:	3 semanas desde la fecha de publicación (Lunes 20 de Abril , 23:59 hrs)
Entrega:	Repositorio GitHub con historial de commits
Ejecución:	`python main.py` debe funcionar sin errores
Restricción:	Prohibido escribir lógica directamente en `main.py`

1. Programación Modular en Python: Arquitectura del Proyecto

Concepto clave – Módulos, Paquetes y la sentencia Import

En proyectos de ingeniería reales, el código no se escribe en un solo archivo interminable. Se divide lógicamente para asegurar su mantenibilidad y escalabilidad:

- Un **módulo** en Python es cualquier archivo `.py` que contiene funciones, clases o variables enfocadas en una tarea específica (código reutilizable).
- Un **paquete** es un directorio que agrupa varios módulos relacionados. Para que Python reconozca este directorio como un paquete importable, debe contener un archivo especial llamado `__init__.py`.

Este concepto de modularidad **no se cubre en clases**; el aprendizaje autónomo es una competencia fundamental en la ingeniería. Deben investigarlo y documentar su comprensión en el archivo `README.md`.

Términos de búsqueda recomendados: *"Python packages tutorial"* y *"Python __init__.py explained"*.

La estructura de directorios que **estrictamente deben respetar** es la siguiente:

```
tarea1_robot_beach/
|-- main.py                 % Punto de entrada: orquesta, no calcula
|-- README.md               % Documentacion: modularidad, dependencias,
   ejecucion
|-- data/
|   |-- __init__.py
|   `-- robot_data.py       % Datos del paper y generacion de senales
|-- processing/
|   |-- __init__.py
|   |-- metricas.py         % Calculo de IAE, ISE, ITAE, ITSE (Ecuaciones
    11-14)
|   `-- cinematica.py       % Modelo cinematico (Ecuaciones 1-6)
|-- visualization/
|   |-- __init__.py
|   `-- graficos.py         % Generacion de figuras con Matplotlib
`-- resultados_graficos/    % Directorio destino para guardar los .png
```

Listing 1: Estructura obligatoria del proyecto

Nota: Para asegurar la correcta integración de los módulos, **se les hará entrega del archivo `main.py` base**. Este archivo ya contiene la estructura de orquestación y las llamadas a las funciones principales.

Su labor como equipo de ingeniería es:

1. Crear la estructura de carpetas y los archivos `__init__.py` correspondientes.
2. Implementar la lógica interna de cada módulo (`robot_data.py`, `metricas.py`, etc.) respetando estrictamente los nombres de las funciones y los parámetros definidos en este enunciado.
3. Asegurarse de que al ejecutar `python main.py`, el sistema sea capaz de importar sus funciones, procesar los datos y generar los gráficos en la carpeta de destino.

2. Partes de la Tarea

Parte 1 – Análisis del Paper – *Entregable teórico sin código*

Como paso previo a la programación, es fundamental comprender el dominio del problema. Antes de escribir una sola línea de código, se deberá entregar un informe conciso (máximo una página) respondiendo a lo siguiente:

1. ¿Qué problema del mundo real motiva el desarrollo de este trabajo? Descríbalo con sus propias palabras.
2. Identifique los sensores equipados en el robot. ¿Cuál es la justificación técnica para elegir estos instrumentos específicos frente a alternativas de mayor costo o complejidad?
3. Explique la diferencia arquitectónica o algorítmica fundamental entre las políticas **PPO** y **PPO-Mask**. ¿Qué ventaja operativa aporta la implementación de una *máscara de acciones*?
4. El documento reporta una reducción de desempeño experimental del 16,6 % en el índice ISE y del 5,2 % en el IAE. En el contexto estricto de la navegación robótica, ¿qué magnitud física o tipo de comportamiento penaliza de mayor manera el índice ISE?

Nota: **Requisito Habilitante:** La entrega de este informe de lectura es obligatoria. Si un grupo omite esta parte, **el código fuente no será evaluado**.

Nota: Para descargar el paper deben acceder a : Paper Guillermo Cid Ampuero

El paper se encuentra en inglés, pero pueden utilizar traductores para cualquier duda.

El informe debe tener como máximo un pagina de extensión y debe ser entregado en formato .PDF (archivos en formatos .word no serán revisados) de la siguiente manera:

```
Tarea 1 : Programacion 2
Intregrantes: Nombre Apellido, Nombre Apellido ....

Pregunta 1 :
Respuesta 1:

Pregunta 2 :
Respuesta 2 :
.
.
.
Pregunta 4:
Respuesta 4 :
```

Parte 2 – Módulo `data/robot_data.py` – Estructuración de Datos

Este módulo es el nucleo del proyecto. Su misión es proveer información estructurada al resto de los módulos. Implementen las siguientes funciones:

(a) `cargar_experimentos()`
Debe retornar un **diccionario de diccionarios** con los resultados de las Tablas 6, 7 y 8 del paper.

- **Estructura obligatoria:** `{ "exp1": {...datos...}, "exp2": {...datos...} }`.
- **Llaves internas:** `politica`, `ambiente`, `ruta`, `ISE`, `IAE`, `ITSE`, `ITAE`, `tiempo_s`, `pasos` y `reward_medio`.
- Use `None` para los campos que no existan en una tabla específica.

Ejemplo:

```
experimentos = {
# Tabla 6: Experimento simple
"exp1": {"politica": "PPO",      "ambiente": "real",       "ruta": "simple",    "ISE": 434.99, "IAE":
     135.93, "ITSE": 6932.79, "ITAE": 2601.61, "tiempo_s": None, "pasos": None, "reward_medio": None},
"exp2": {"politica": "PPO-Mask", "ambiente": "real",       "ruta": "simple",    "ISE": 362.85, "IAE":
     128.92, "ITSE": 5869.30, "ITAE": 2669.86, "tiempo_s": None, "pasos": None, "reward_medio": None},
     .... }
```

El diccionario tendrá 12 claves , donde cada clave tendra como valor un diccionario con 10 claves, como se muestra en el ejemplo anterior. Todos los datos deben ser extraídos de las **Tablas 6,7 y 8**

(b) `generar_trayectoria_ideal(waypoints, puntos_por_segmento=100)`
El argumento waypoints recibe en el archivo main.py una lista de listas con coordenadas (ej. `[[0,0], [4,0], [4,4]]`). Su función debe:

1. Cree dos listas vacías: `x_ideal` e `y_ideal`.
2. Iteren sobre los pares de puntos consecutivos de los `waypoints`.
3. Usen `np.linspace` para generar los puntos intermedios de cada segmento y agréguenlos a sus listas usando el método `.extend()` (Investiga con utilizar .extend() de numpy).
4. **Retorno:** Los dos arreglos de NumPy: `np.array(x_ideal)` y `np.array(y_ideal)`.

(c) `simular_lidar(n_sectores=36, d_min=0.5, d_max=30.0)`
Genera una lectura simulada del sensor RPLIDAR S2.

1. Cree un arreglo de ángulos de 0 a 360° usando `np.linspace`.
2. Genere distancias aleatorias uniformes entre `d_min` y `d_max`.
3. **Simulación de Obstáculo:** Modifique los índices del `[5:9]` para que tengan distancias pequeñas (entre 0.5 y 2.0 m), simulando un objeto cercano al robot.
4. **Normalización:** Calcule un tercer arreglo `distancias_norm` aplicando la fórmula:

$$d_{norm} = \frac{d - d_{min}}{d_{max} - d_{min}}$$

Retorne: `angulos_deg`, `distancias` y `distancias_norm`.

Parte 3 – Módulo `visualization/graficos.py` – Comunicación Visual

La comunicación visual de resultados es vital en ingeniería. En este módulo, deben transformar los datos numéricos en información comprensible y automatizar el guardado de reportes.

Concepto clave – Gestión de Archivos con la Librería `os`

Para que su código sea robusto, no debe depender de rutas escritas a mano (ej. `"C:/Users/..."`). Python utiliza la librería estándar `os` para interactuar con el sistema operativo.

- **Investigación obligatoria:** Busquen cómo utilizar `os.makedirs(folder, exist_ok=True)` para crear carpetas automáticamente y `os.path.join()` para unir nombres de carpetas y archivos de forma segura.
- **Matplotlib:** Investiguen la función `plt.savefig()` y cómo configurar el parámetro `dpi` para controlar la calidad de la imagen guardada.

Implementen las siguientes funciones en su módulo:

(a) `plot_metricas(diccionario_experimentos, ambiente, ruta)`
Esta función debe generar una figura con **4 subplots lineales** (1 fila, 4 columnas) usando `plt.subplot()`. Esta función, se utiliza en el código main.py, donde tiene como argumento de entrada el diccionario de diccionarios creada en el módulo anterior **cargar_experimentos()**

1. **Filtrado:** Use `.items()` para recorrer el diccionario de experimentos y extraiga solo los datos que coincidan con el `ambiente` y `ruta` solicitados.
2. **Gráficos:** Cada subplot debe mostrar un gráfico de barras (`plt.bar`) comparando PPO vs PPO-Mask para una métrica (ISE, IAE, ITSE, ITAE).
3. **Automatización:** Use la librería `os` para asegurar que la carpeta `resultados_graficos/` exista antes de guardar la figura con un nombre descriptivo.

(b) `plot_lidar(angulos, distancias, distancias_norm)`
Visualice la percepción del robot para depurar posibles colisiones. Esta función, se utiliza en el código main.py, tiene como argumento de entrada los valores que entrega la función **simular_lidar()**.

- **Subplot 1 (Real):** Use `plt.scatter()` para mostrar las distancias en metros. Ideal para que el operador humano vea dónde están los obstáculos.
- **Subplot 2 (IA):** Use `plt.plot()` para las distancias normalizadas. Este es el vector que "lee" la red neuronal.

(c) `plot_trayectorias(x_ppo, y_ppo, x_mask, y_mask, waypoints, nombre)`
Genere el mapa de navegación comparativo:

- Dibuje las trayectorias con `plt.plot()` y los `waypoints` como cuadrados negros con `plt.scatter(..., marker='s')`.
- **Restricción Geométrica:** Es obligatorio el uso de `plt.axis('equal')` para respetar la escala física de la playa (1 metro en X debe medir lo mismo que 1 metro en Y).

Parte 4 – Módulo `processing/metricas.py` – Evaluación del Controlador

En este módulo, se implementarán las métricas que permiten decir científicamente si un robot es mejor que otro.

Concepto clave – Índices de error de control (Ecs. 11–14 del paper)
Sea $e(i)$ el error de posición en el paso i. En software, aproximamos las integrales del paper a sumatorias discretas multiplicadas por el paso de tiempo Δt (dt):

- **IAE** (Integral Absolute Error): $\sum |e_i| \cdot dt$
- **ISE** (Integral Square Error): $\sum e_i^2 \cdot dt$
- **ITAE/ITSE**: Son versiones que multiplican el error por el tiempo ($t_i = i \cdot dt$), castigando los errores que tardan mucho en corregirse.

Regla de Oro: Mientras menor sea el valor, más "inteligente" es el robot.

Implementen las siguientes funciones utilizando **NumPy** (está prohibido usar ciclos `for` para las sumatorias, usen `np.sum`):

(a) `calcular_IAE, calcular_ISE, calcular_ITAE, calcular_ITSE`
Cada función recibe el arreglo de `errores` y el escalar `dt`. Deben retornar un `float`. Para las métricas con tiempo (IT), generen el arreglo de tiempo usando `np.arange(len(errores)) * dt`.

(b) `calcular_todas_las_metricas(errores, dt)`
Función agrupadora que llama a las 4 anteriores y retorna un diccionario con las llaves: `"ISE", "IAE", "ITSE", "ITAE"`. Use la función `round(valor, 2)` para entregar resultados limpios.

(c) `calcular_mejora(valor_ppo, valor_mask)`
Calcula la reducción porcentual del error: $\frac{V_{base}-V_{nuevo}}{V_{base}} \times 100$.

Esta función permite verificar si el PPO-Mask realmente mejora al PPO estándar como dice el paper (16.6 % en ISE).

Parte 5 – Módulo `processing/cinematica.py` – Física del Robot

En esta sección se programará las leyes físicas que rigen el movimiento del robot diferencial.

Concepto clave – Modelo Cinemático y Error Espacial

El robot se mueve siguiendo: $x_{new} = x + v \cdot \cos(\theta) \cdot dt$. Para evaluar el desempeño, comparamos la posición real del robot (x_r, y_r) contra la posición ideal (x_i, y_i) que debería tener en ese instante.

(a) `calcular_movimiento(x, y, theta, v, omega, dt=0.1)`
Calcula la nueva pose del robot.

- **Saturación:** Antes de calcular, use `np.clip` para asegurar que v no supere los 0.8 m/s y ω los 0.6 rad/s (Restricciones de la Tabla 1).
- **Retorno:** La tupla `(x_nuevo, y_nuevo, theta_nuevo)`.

(b) `distancia_al_objetivo(x, y, x_meta, y_meta)`
Retorna la distancia euclidiana (usando `np.sqrt`) entre el robot y el punto al que quiere llegar. Esta función es vital para que el `main.py` decida cuándo pasar al siguiente *waypoint*.

(c) `calcular_error_seguimiento(x_real, y_real, x_ideal, y_ideal)`
Recibe los arreglos completos de la trayectoria que hizo el robot y la que debió hacer. Retorna un arreglo de NumPy con la distancia de error en cada punto. *Pista:* Si los arreglos tienen distinto tamaño, use el tamaño del más corto para evitar errores de índice.

3. Guía de Integración: Desglosando el Orquestador `main.py`

Para asegurar que el proyecto funcione como un sistema integrado, el equipo docente les entregará el archivo `main.py`. A continuación, se explica bloque por bloque la lógica de este Director de Orquesta, el cual deben estudiar para entender cómo sus módulos se conectan entre sí.

3.1. 1. Importación y Dependencias

En la cabecera del script, realizamos las importaciones de las librerías estándar (`os`, `numpy`) y, lo más importante, de **sus propios módulos**.

- `from data.robot_data import ...`: Carga las funciones provenientes desde `data/robot_data.py`.
- `from processing....`: Carga las funciones provenientes desde el código `processing/cinematicas.py` y `processing/metricas.py`.
- `from visualization.graficos import ...`: Carga las funciones encargadas de realizar los gráficos, desde `visualization/graficos.py`.

3.2. 2. El Flujo de Ejecución (*The Pipeline*)

El código dentro de la función `main()` sigue un orden lógico secuencial:

Paso 1: Carga de Datos Estáticos
Se ejecuta `cargar_experimentos()`. Aquí, Python busca en su archivo `robot_data.py` el diccionario de diccionarios con las Tablas del paper. Si este paso falla con un `KeyError`, revisen que las llaves internas (`"ISE"`, `"politica"`, etc.) estén escritas exactamente como se pidió.

Paso 2: Validación Cinemática (Prueba de Motores)
Antes de procesar grandes volúmenes de datos, el sistema realiza una prueba unitaria del modelo físico. Se invoca a `calcular_movimiento()` con una pose inicial en el origen $(0, 0, 0)$ y velocidades de prueba. Luego, se utiliza `distancia_al_objetivo()` para verificar qué tan lejos quedó el robot de una meta arbitraria. Si los valores impresos en consola no tienen sentido físico, el error se encuentra en su implementación de las Ecuaciones 1-6. Para los valores de test entregados en main.py

```
x_test, y_test, th_test = 0.0, 0.0, 0.0
v_test, w_test = 0.5, 0.1  # Velocidades de prueba

# Probamos el movimiento unitario
x_n, y_n, th_n = calcular_movimiento(x_test, y_test, th_test, v_test
    , w_test)
# Verificamos que tan lejos quedo de una meta arbitraria (1,1)
dist_test = distancia_al_objetivo(x_n, y_n, x_meta=1.0, y_meta=1.0)
```

Se debe obtener:

```
> Pose inicial: (0,0,0) -> Nueva Pose: (0.05, 0.00, 0.01 rad)
> Distancia restante al objetivo (1,1): 1.38m
```

Paso 3: Simulación Sensorial (LiDAR)
El sistema genera ángulos y distancias aleatorias con la función **simular_lidar**. Luego, llama a su función `plot_lidar()` para realizar el gráfico de los datos. Este paso debería entregarles como resultados la imagen `mapa_lidar.png` (o `mapa_lidar_basico.png`) similar a la que se observa a continuación:

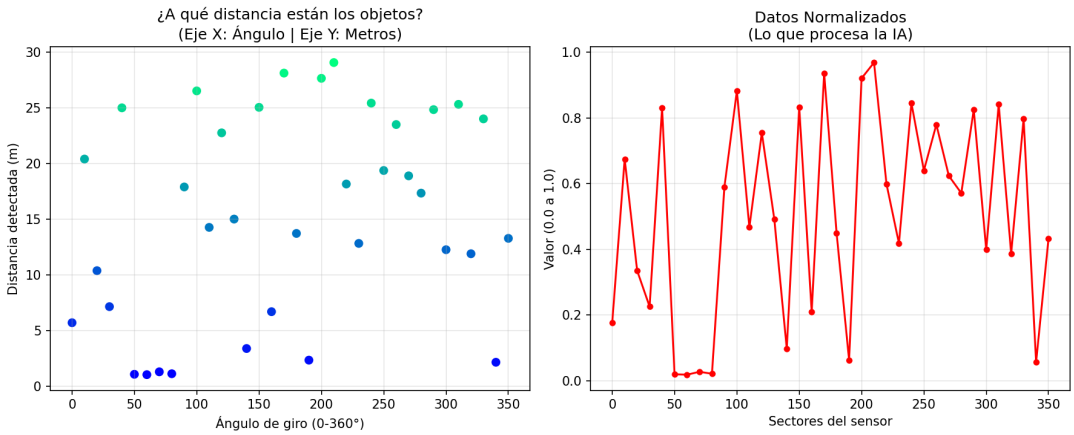

Figura 1: Imagen esperada en Paso 3 (Visualización LiDAR)

Paso 4: Pruebas de Navegación (Triángulo y Cuadrado)
Este es el bloque más robusto. Para cada ruta:

1. El código `main.py` genera los puntos ideales que el robot **debería** seguir.
   - `ruta_tri = [[0, 0], [4, 0], [2, 4], [0, 0]]`
   - `ruta_cuad = [[0, 0], [4, 0], [4, 4], [0, 4], [0, 0]]`
2. Se crean dos trayectorias "reales" ficticias usando `np.random.normal`.

```
        x_ppo_tri = x_id_tri + np.random.normal(0, 0.12, len(x_id_tri))
        y_ppo_tri = y_id_tri + np.random.normal(0, 0.12, len(y_id_tri))
        x_mask_tri = x_id_tri + np.random.normal(0, 0.04, len(x_id_tri))
        y_mask_tri = y_id_tri + np.random.normal(0, 0.04, len(y_id_tri))
```

3. La trayectoria de `PPO-Mask` tiene una desviación menor (**0.04 - 0.05**) que la de `PPO` (**0.12 - 0.15**). Esto permite visualizar gráficamente por qué una política es superior a la otra antes de ver los números finales en el gráfico `trayectorias_triangulo.png`.

Luego se realiza la gráfica de la ruta triangular y la ruta cuadrada, para ambas , si se realizó la correcta programación de la función de trayectorias, se debe obtener una imagen como la que observaremos a continuación:

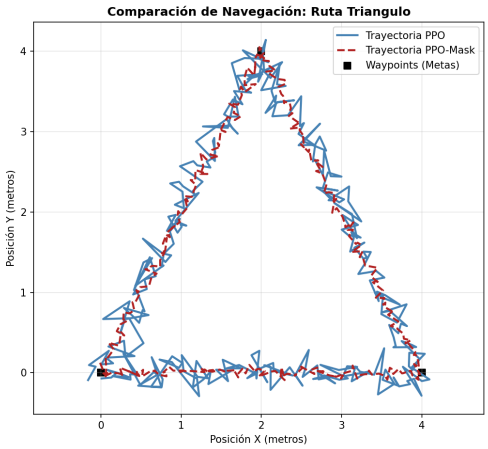

Figura 2: Resultado Ruta Triangular

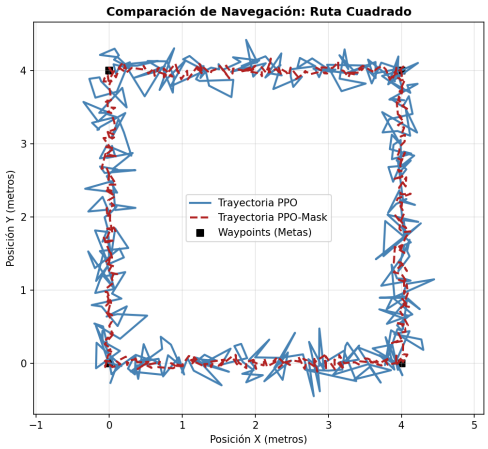

Figura 3: Resultado Ruta Cuadrada

Paso 5: Análisis de Desempeño Matemático
Aquí es donde se utiliza el módulo `metricas.py` . El `main` toma la trayectoria simulada del robot en la ruta cuadrada y la compara contra la ideal usando `calcular_error_seguimiento()`. El resultado es una serie de errores que `calcular_todas_las_metricas()` transforma en los índices ISE, IAE, ITSE e ITAE. Estos valores se imprimen en la consola para su análisis. Si se realizó de manera correcta el código de metricas.py, se debe obetener como resultado en el main.py :

```

[METRICAS] Resultados PPO en Ruta Cuadrada:
 > ISE: 1.97
 > IAE: 7.83
 > ITSE: 39.13
 > ITAE: 155.01
```

Paso 6: Reporte Final del Paper
Finalmente, se llama a `plot_metricas()` pasando el diccionario de experimentos. Esto generará el gráfico de 4 subplots de barras comparativas que ustedes deben incluir obligatoriamente en su informe final. El resultado de este último plot debe verse como la imagen a continuación:

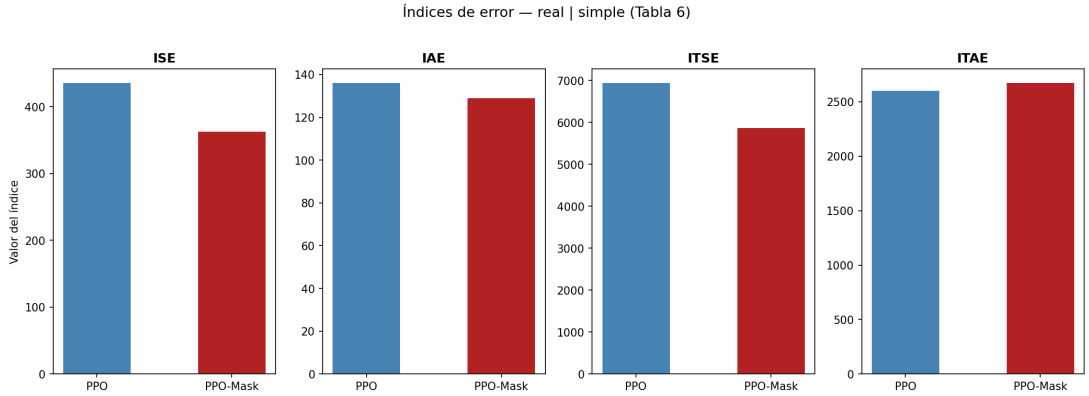

Figura 4: Resultado Plot Métricas

Nota:
El código main.py que se entregado **No deben modificarlo**, salvo que necesiten agregar impresiones extra para depurar su código (*debugging*).

4. Rúbrica de Evaluación

Parte	Descripción	Puntaje
0	Análisis escrito del paper	10 %
1	Módulo `robot_data.py`	20 %
2	Módulo `metricas.py`	20 %
3	Módulo `cinematica.py`	20 %
4	Módulo `graficos.py` (4 figuras guardadas)	20 %
5	`main.py` funcional + reporte en consola	10 %
	Total	**100 %**

Nota: Material de apoyo

- https://github.com/guille-robotics/Programacion_2_EIE_434.git
- Material del Aula Virtual
- https://ellibrodepython.com/